

Assignment 2: AST Generator for MiRust

Version: v1.0

1. OBJECTIVES

- Learn how to build an Abstract Syntax Tree (AST) from an ANTLR4 parse tree using the Visitor pattern.
- Understand how to map concrete syntax structures to abstract syntax nodes.
- Handle type annotations, complex expressions (such as right-associative assignments, logical operators, and unary operations), structures, loops, and control statements.
- Build a Python testing harness to run the AST generator tests.

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
 - Read `src/utils/AST.py` to familiarize yourself with the classes representing the AST nodes.
- **Task 2: Implement the ASTGenerator.py file**
 - Complete the class `ASTGenerator` in `src/astgen/ASTGenerator.py` (inheriting from `MiRustVisitor`).
 - Implement visitor methods for each parser rule to traverse the parse tree and return the corresponding AST representation.
- **Task 3: Write 100 AST generator tests**
 - Create 100 test cases in `test/test-ast.py` (`test_1` to `test_100`) using the provided `check_ast` helper.
 - Test precedence, associativity, literal string escaping, control flow statements, variable and function declarations, array operations, and structure definitions/initializations.

3. AST NODE MAPPING SPECIFICATION

Your visitor must return AST instances matching the structure defined in `src/utils/AST.py` :

1. Declarations:

- `Program(decls: list[Decl])` : root of the AST containing a list of structure declarations followed by function declarations.
- `StructDecl(name: Id, fields: list[VarDecl])` : represents `struct ID { field1, field2, ... }`. Note that structure fields are modeled as `VarDecl` with `init=None` and `is_mut=False`.
- `FuncDecl(name: Id, params: list[VarDecl], return_type: Type, body: Block)` : function definition.
- `TypeAliasDecl(name: Id, origin_type: Type)` : represents type aliasing `type alias = origin_type`.

2. Types:

- `IntType`, `FloatType`, `BoolType`, `StringType`, `VoidType`.
- `ArrayType(element_type: Type, size: int)`.
- `NamedType(name: Id)`.
- `BoxType(element_type: Type)`.

3. Statements:

- `Block(stmts: list[Stmt])` : a block of statements enclosed in curly braces.
- `VarDecl(name: Id, var_type: Type | None, init: Expr | None, is_mut: bool)` : local variable declaration.
- `If(cond: Expr, then_stmt: Stmt, else_stmt: Stmt | None)` : if/else branch structure.
- `While(cond: Expr, body: Stmt)` : while loop.
- `For(var_name: Id, start: Expr, end: Expr, body: Stmt)` : range-based for loop.
- `Switch(expr: Expr, cases: list[Case], default_stmt: list[Stmt] | None)` : switch statement with cases and an optional default block.
- `Case(val: Expr, body: list[Stmt])` : single case clause.
- `Break()` , `Continue()` .
- `Return(expr: Expr | None)` .
- `ExprStmt(expr: Expr)` : statement wrapping an expression.

4. Expressions:

- `Assign(lhs: Expr, rhs: Expr)` : right-associative assignment expression (returns the value of right-hand side in MiRust).
- `BinaryOp(op: str, left: Expr, right: Expr)` : binary operations (arithmetic, relational, logical).
- `UnaryOp(op: str, body: Expr)` : unary operations (`-` , `!` , `*`).
- `CallExpr(name: Id, args: list[Expr])` : function call expression (also used for `Box_new`).
- `ArrayCell(arr: Expr, idx: Expr)` : indexing into an array (e.g. `arr[idx]`).
- `FieldAccess(obj: Expr, field: Id)` : accessing a structure field (e.g. `obj.field`).
- `StructInit(name: Id, initializers: list[FieldInit])` : struct instantiation (e.g. `Point { x: 1, y: 2 }`).
- `FieldInit(name: Id, expr: Expr)` : single field initializer in a struct instantiation.
- `Id(name: str)` : variable/identifier reference.

5. Literals:

- `IntLiteral(value: int)` .
- `FloatLiteral(value: float)` .
- `BoolLiteral(value: bool)` .
- `StringLiteral(value: str)` : string literal containing the evaluated text (with backslash escape sequences like `\n` or `\t` decoded into their respective characters).
- `ArrayLiteral(elements: list[Expr])` .
- `NilLiteral()` : nil literal.

4. REQUISITES & SETUP

- Compile the grammar for Python target: `python src/run.py gen`
- Execute your unit tests: `python src/run.py test-ast`

5. PROJECT STRUCTURE

The repository layout for Assignment 2 must be structured as follows:

```

root/
├── src/
│   ├── grammar/
│   │   ├── MiRust.g4          # Student's ANTLR4 grammar file
│   │   └── (generated files) # Generated by ANTLR (Lexer, Parser, etc.)
│   ├── astgen/
│   │   └── ASTGenerator.py    # Student's AST generator implementation
│   └── utils/
│       ├── AST.py            # AST node definitions (provided)
│       └── lexerror.py       # Lexer error utilities (provided)
├── test/
│   └── test-ast.py           # Student's 100 AST unit tests
└── requirements.txt

```

6. EVALUATION CRITERIA & GRADING

- **AST Generation Correctness (100%):** Grade is based on passing the automated test cases run against your submitted grammar and `ASTGenerator.py` implementation.
- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-ast.py` is mandatory. A penalty of 10% of the total assignment grade will be deducted if `test/test-ast.py` is missing or contains fewer than 100 test cases.

Test Distribution Table

The 100 tests in `test/test-ast.py` are distributed by language feature as follows to guide your implementation:

Structure	Test Cases	Description
Declarations & Definitions	test_1 - test_15	Program layout, function declarations (<code>void/typed</code>), struct definitions, type aliases, variable declarations (<code>let/let mut</code>), type inference, global/local variables.
Advanced Return Types & Box Functions	test_16 - test_25	Functions returning <code>Box<T></code> , array pointers, nested box types (<code>Box<Box<i32>></code>), and box parameters.
Control Flow Structures & Nested Blocks	test_26 - test_40	Conditional branching (<code>if-else</code>), loop statements (<code>while, for-in</code>), jump statements (<code>break, continue, return</code>), deeply nested blocks <code>{ }</code> .
Switch Statements	test_41 - test_45	C-style switch branching, case arms with/without blocks, fall-through sequences, and default clauses.
Arrays & Struct Instantiation	test_46 - test_55	Array types (<code>[i32; 5]</code> , multidimensional arrays (<code>[i32; 2]; 3]</code>), struct initialization, array literals, and array/field member updates.
Function Calls & Parameters	test_56 - test_65	Function invocations with zero, single, or multiple arguments, nested function calls, and function calls in expressions.

Operators & Precedence	test_66 - test_85	Unary (-, !, *), Multiplicative (*, /, %), Additive (+, -), Relational/Equality (<, <=, >, >=, ==, !=), and Logical (&&,).
Nested Complex Accesses	test_86 - test_96	Deeply nested combinations of array indexing, struct field accesses, box dereferences, and call returns (e.g. <code>arr[matrix[0][1]], *f().arr[1]</code>).
Chained Assignment Expressions	test_97 - test_100	Right-associative assignment chains (<code>x = y = 5</code>), assignments in conditions, and member assignment targets (<code>a.b = c.d = 1</code>).

7. DELIVERABLES

Students must submit:

- File `src/grammar/MiRust.g4` containing the ANTLR4 grammar definition.
- File `src/astgen/ASTGenerator.py` containing the AST generation visitor.
- File `test/test-ast.py` containing exactly 100 AST test cases.

8. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 2.

9. CHANGELOG